



What agentic development means for Rust (and vice versa)

Nicholas D. Matsakis / Sr. Principal Engineer, Amazon

Who is this guy?

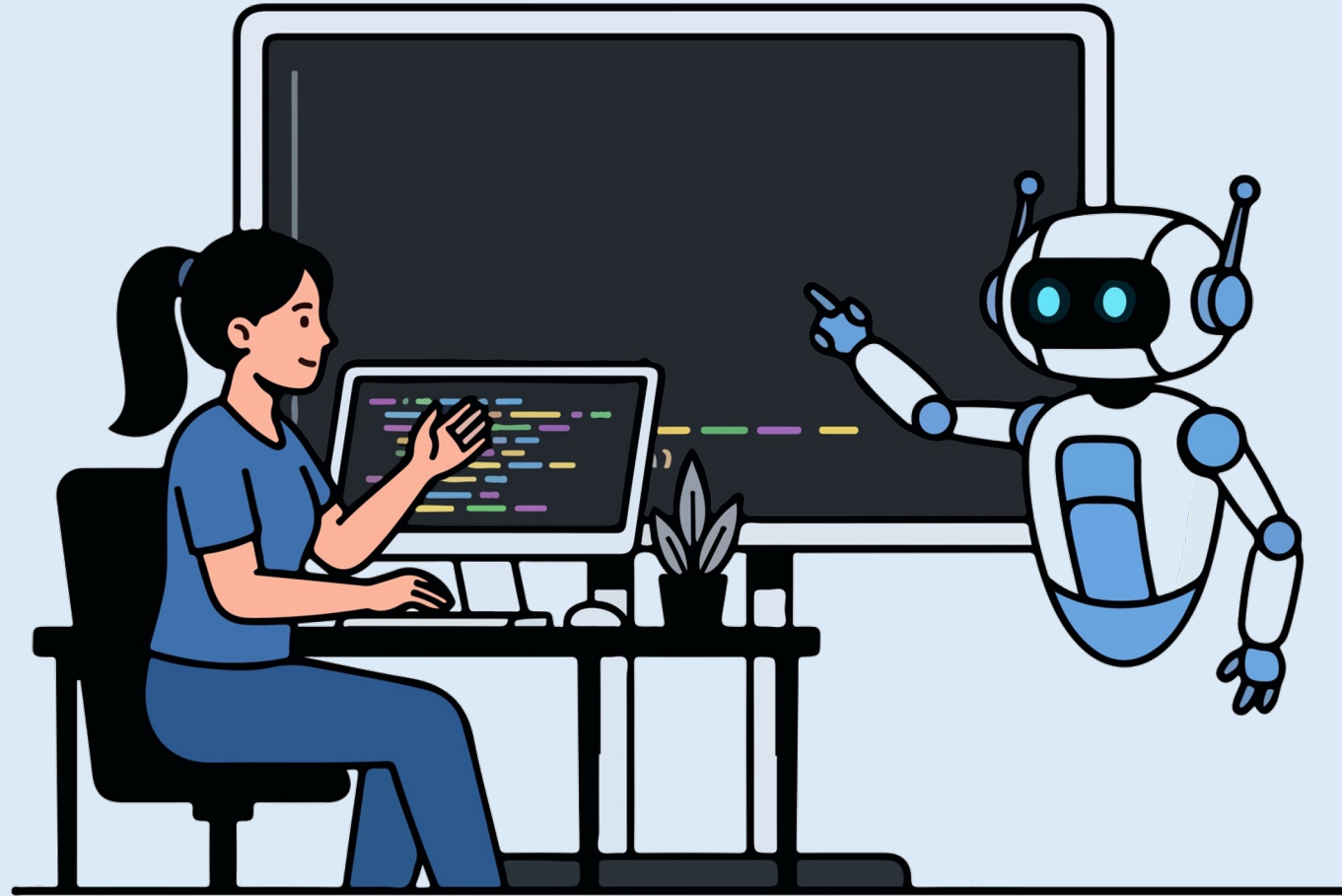
So how do I feel, given the rise of agentic coding?



Programming then



Programming now?





Elon Musk  
@elonmusk



This is exactly right. Source code is on the verge of becoming like assembly.

The next step is getting rid of "source code" entirely and just making an efficient binary directly with AI.



Egads!



I might never look the source again. It was nearly a half century ago that I stopped looking at assembly because I trusted the compiler to get it right. This feels like that.

5:43 PM · Aug 3, 2026 · **8.7M** Views

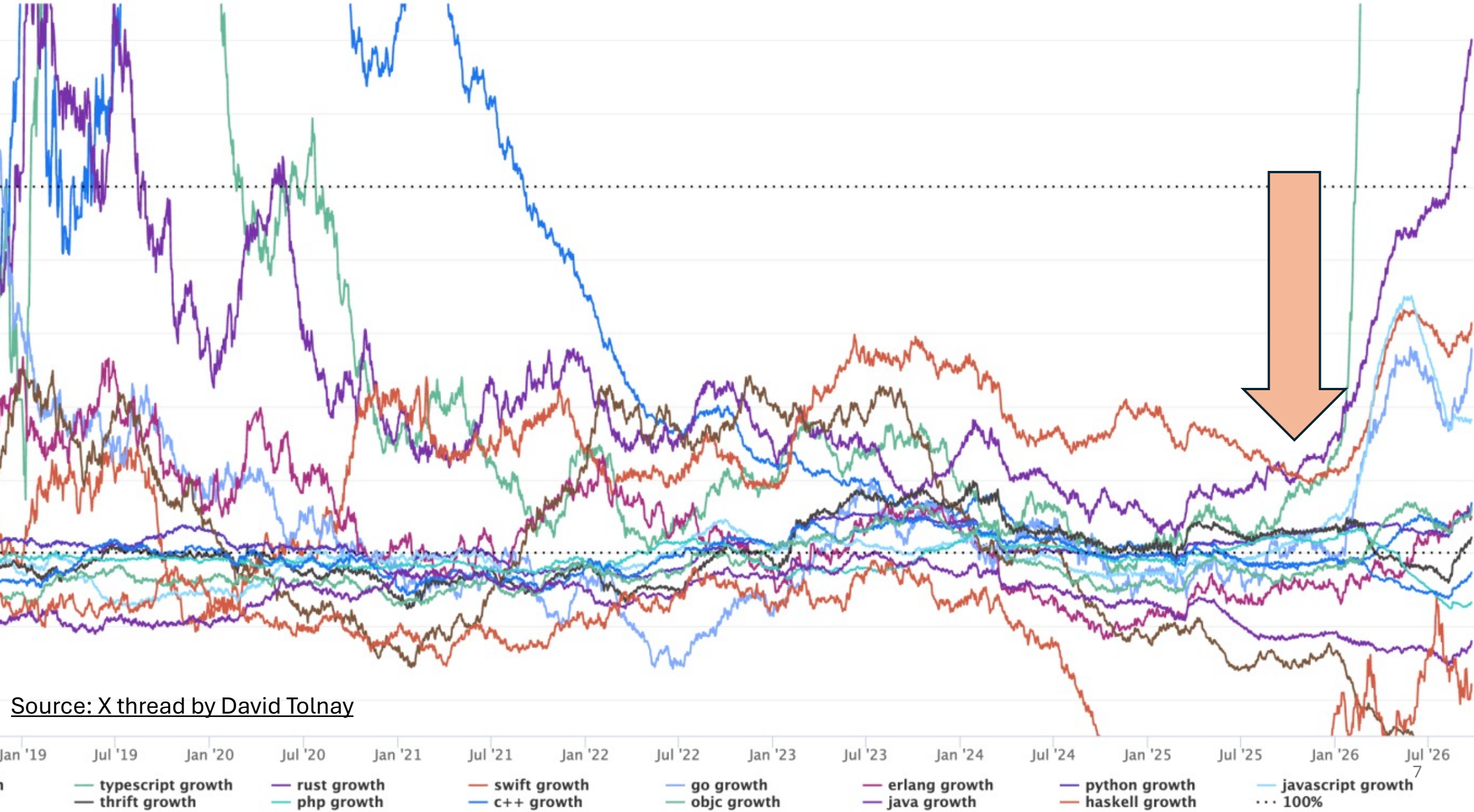
 3.1K

 3K

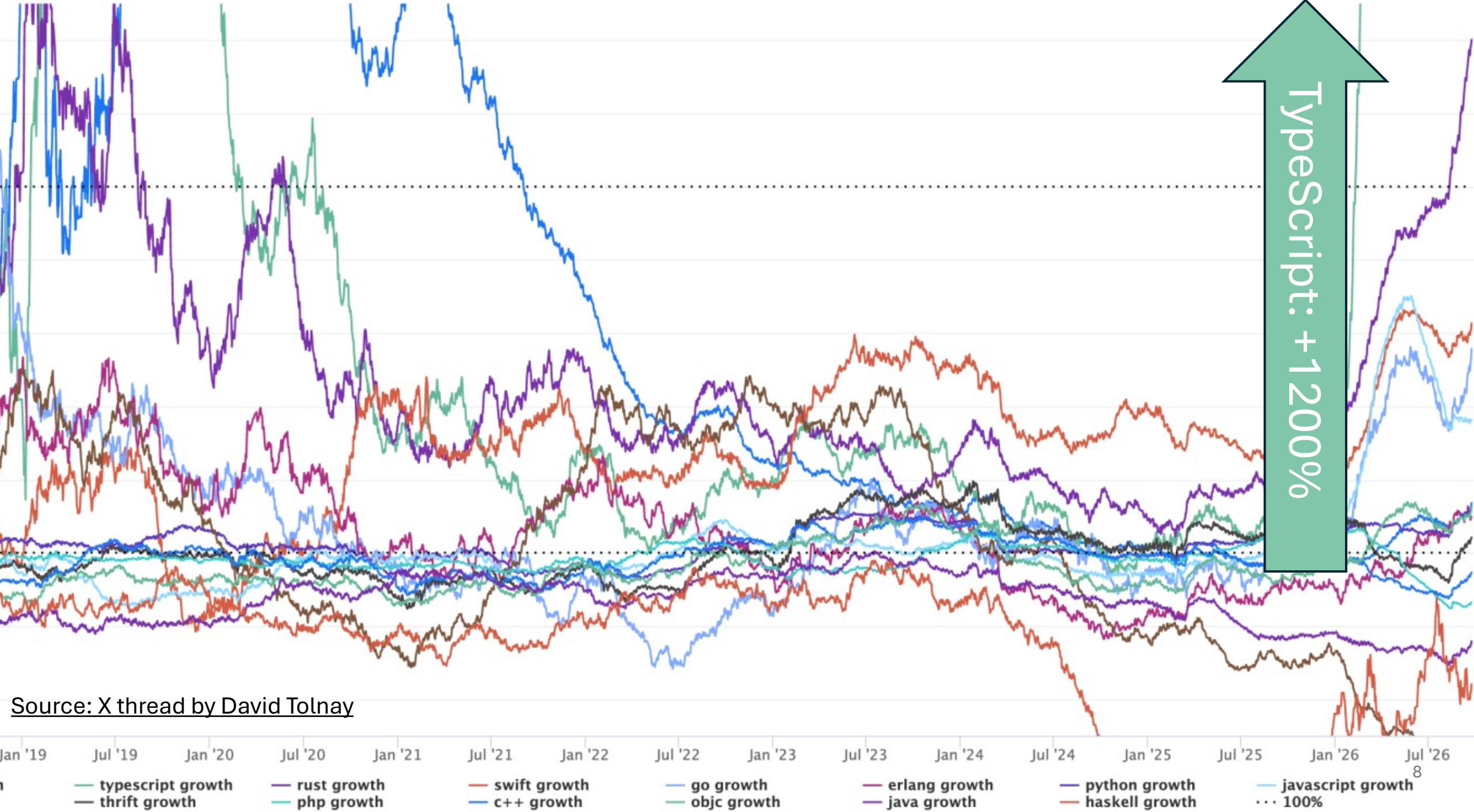
 27K

 4.1K



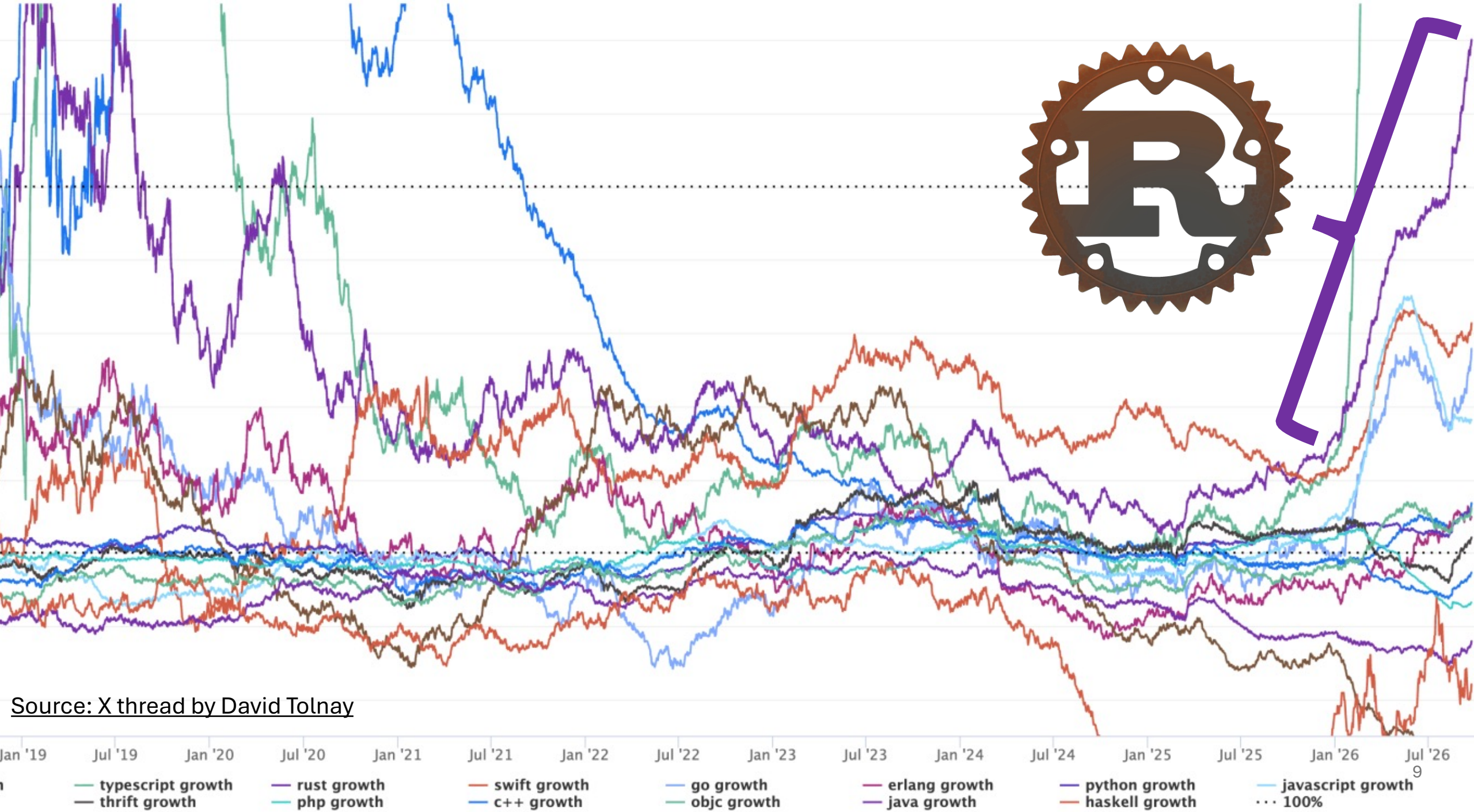


Source: X thread by David Tolnay

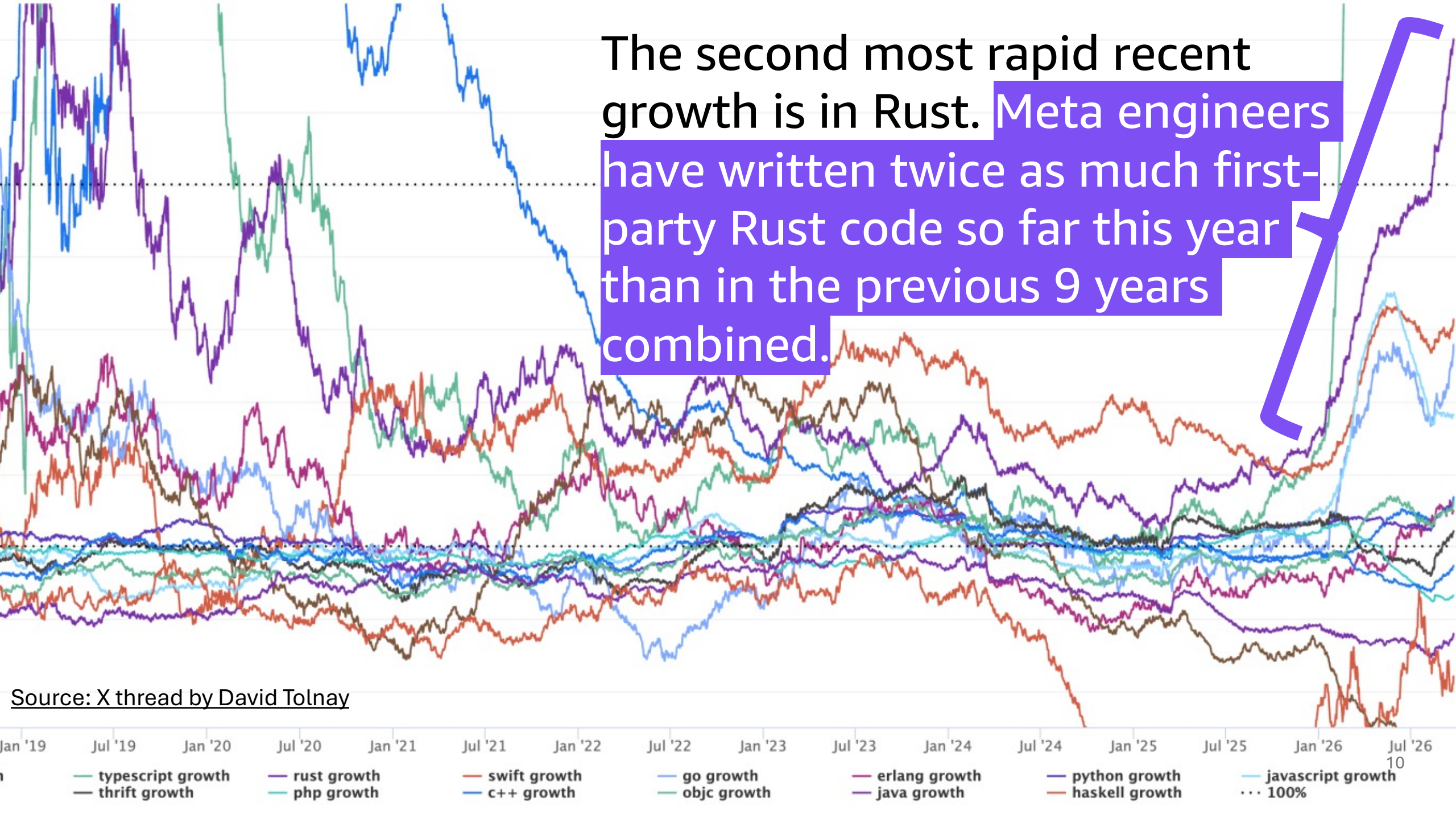


TypeScript: +1200%

Source: X thread by David Tolnay



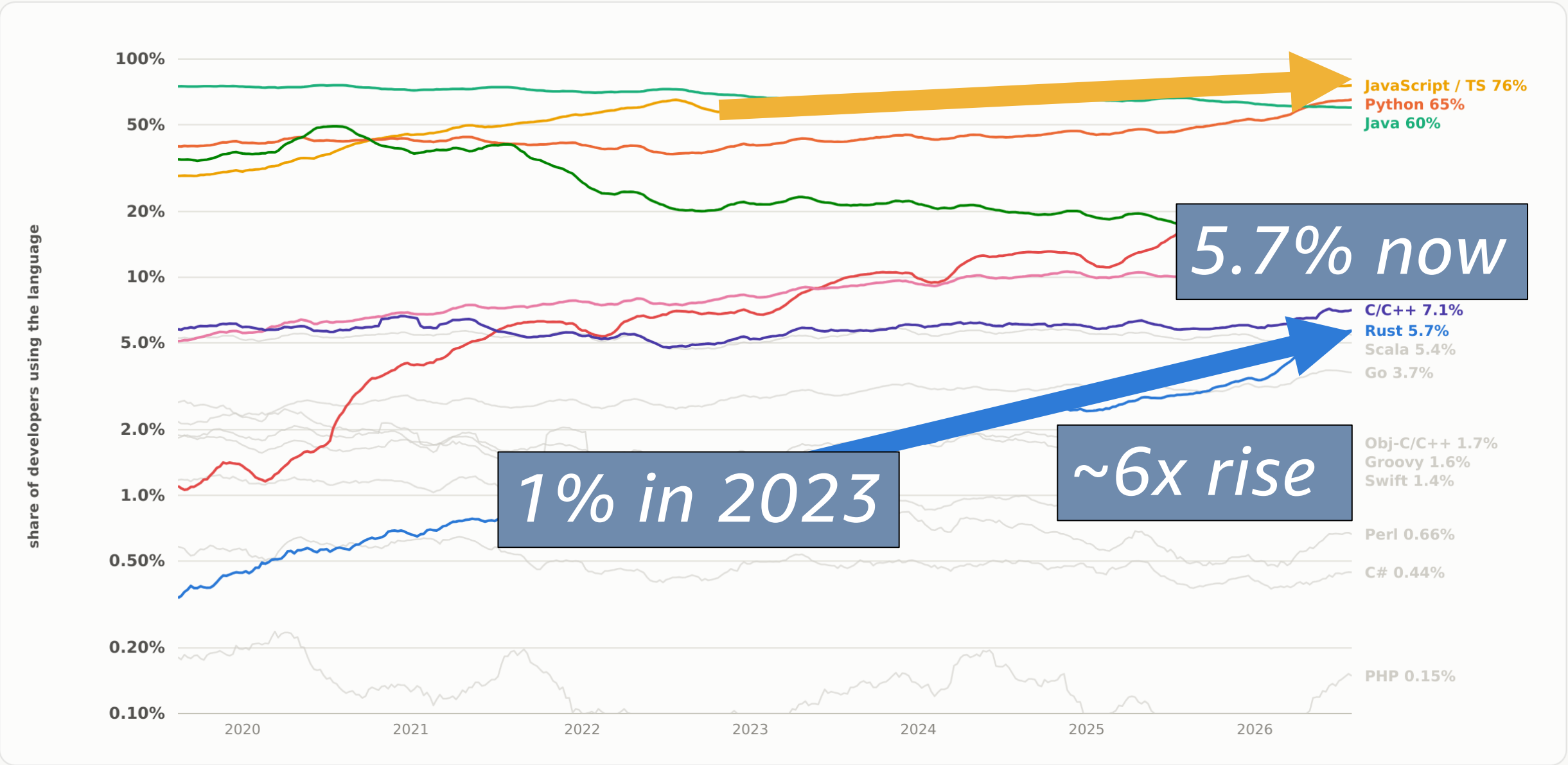
Source: X thread by David Tolnay



Source: X thread by David Tolnay

Language Use at Amazon: Share of developers

The percentage of Amazon's developers who commit in each language, in a rolling 90-day window — log scale, so equal vertical distance is equal multiplication



ME:



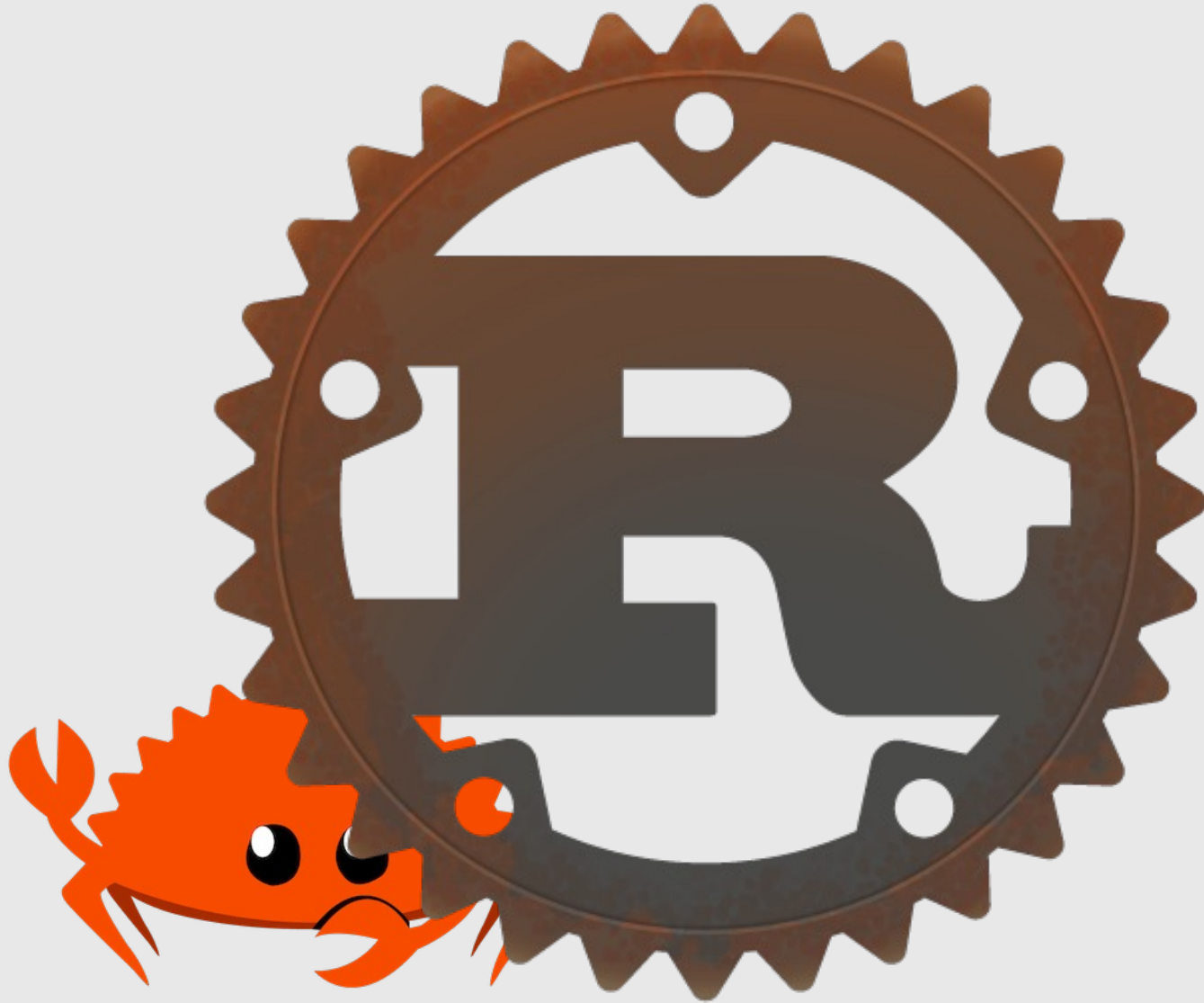
But why?



Be honest.

A man with dark hair and a beard, wearing a dark shirt, looking down and slightly to the side with a serious expression.





“Rust has a lot of the properties you want for agentic coding a lot of the time.”

Me, a totally unbiased observer

What makes Rust a good fit for agentic development?

01 Knobs

02 Guardrails

“I have a Python program I run every day, it takes 1.5 seconds. I spent six hours re-writing it in Rust, now it takes 0.06 seconds. That efficiency improvement means I'll make my time back in 41 years, 24 days :-)”

Twitter post quoted in ["Software Disenchantment"](#) by Nikita Prokopov



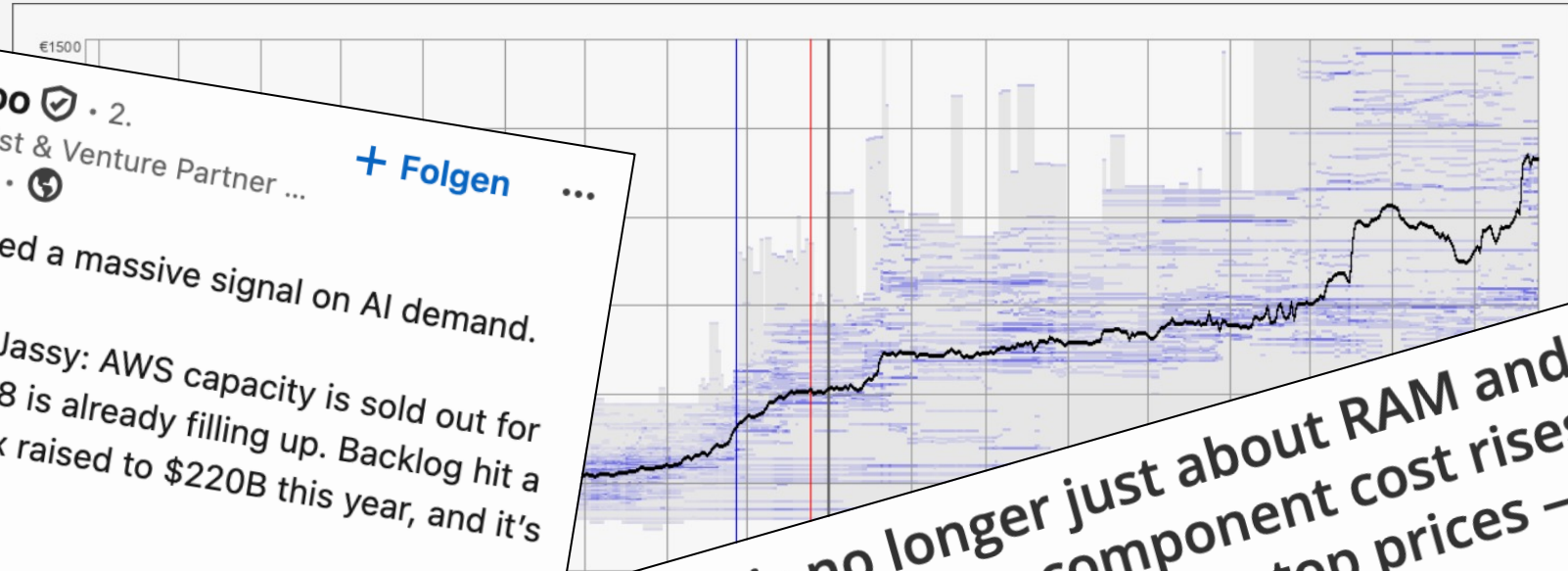


**"A billion [cycles] here, a billion [cycles] there,
and pretty soon you're talking real money!"**

With apologies to Senator Everett McKinley Dirksen

A gigabyte here, a gigabyte there...

DDR4-3600 2x32GB (Average price in EUR over last 18 months)



Alvin Foo ✓ · 2.

AI Strategist & Venture Partner ...
1 Monat(e) · 🌐

+ Folgen ...

Amazon just dropped a massive signal on AI demand.

Amazon CEO Andy Jassy: AWS capacity is sold out for 2026 and 2027, 2028 is already filling up. Backlog hit a record \$496B. Capex raised to \$220B this year, and it's still not enough.

This is the clearest proof yet that the AI infrastructure supercycle is real and supply-constrained. Locked-in multi-year demand, pricing power, and Amazon still leaning hard into buildout despite the near-term cash flow hit.

Risks exist, but moments like this separate the durable trends from the noise.

'This is no longer just about RAM and SSDs': XMG highlights component cost rises across the board as it hikes laptop prices — and Nvidia GPUs could suffer too

News

By Darren Allan | Published 2 days ago

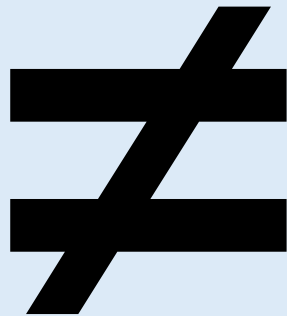
DDR5 memory for notebooks is now close to 6x the price it was before the RAM crisis

TECH RADAR

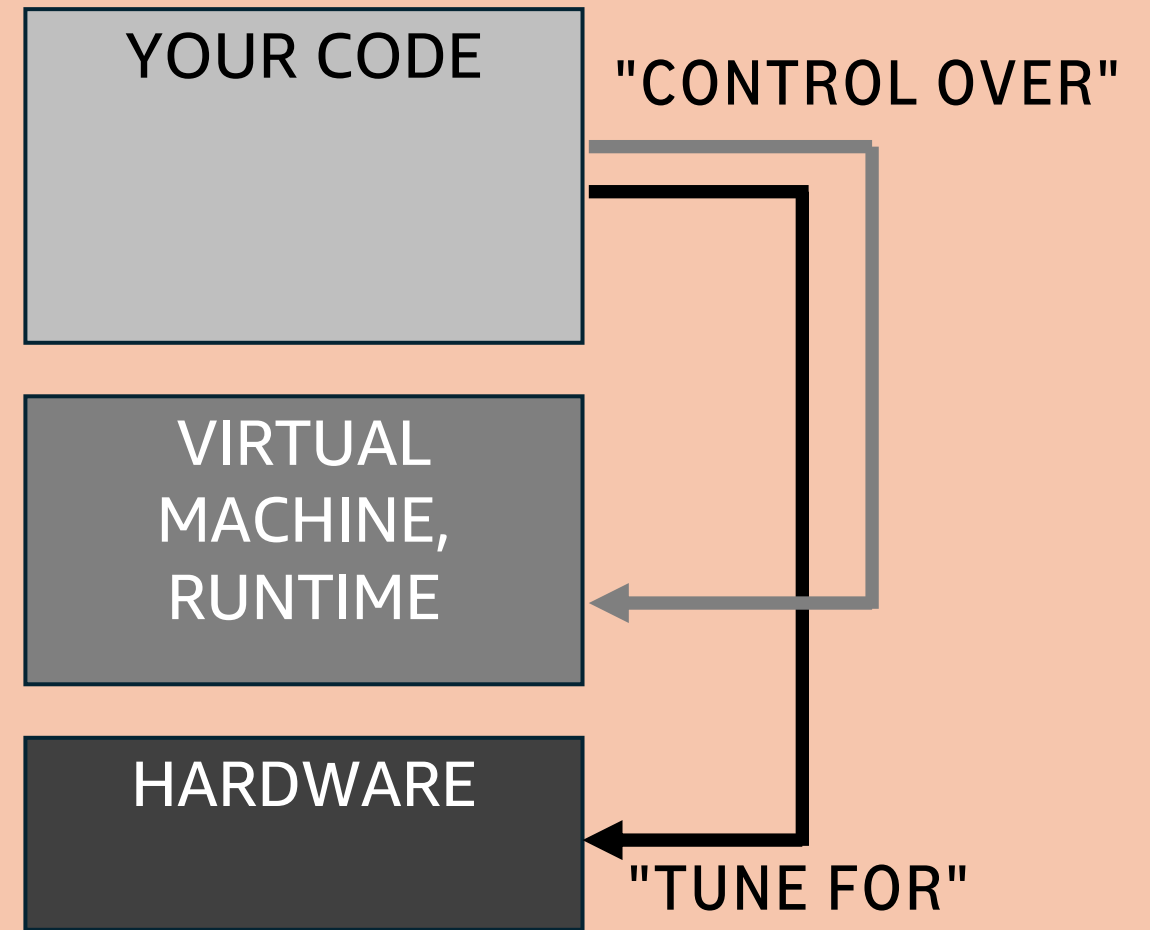
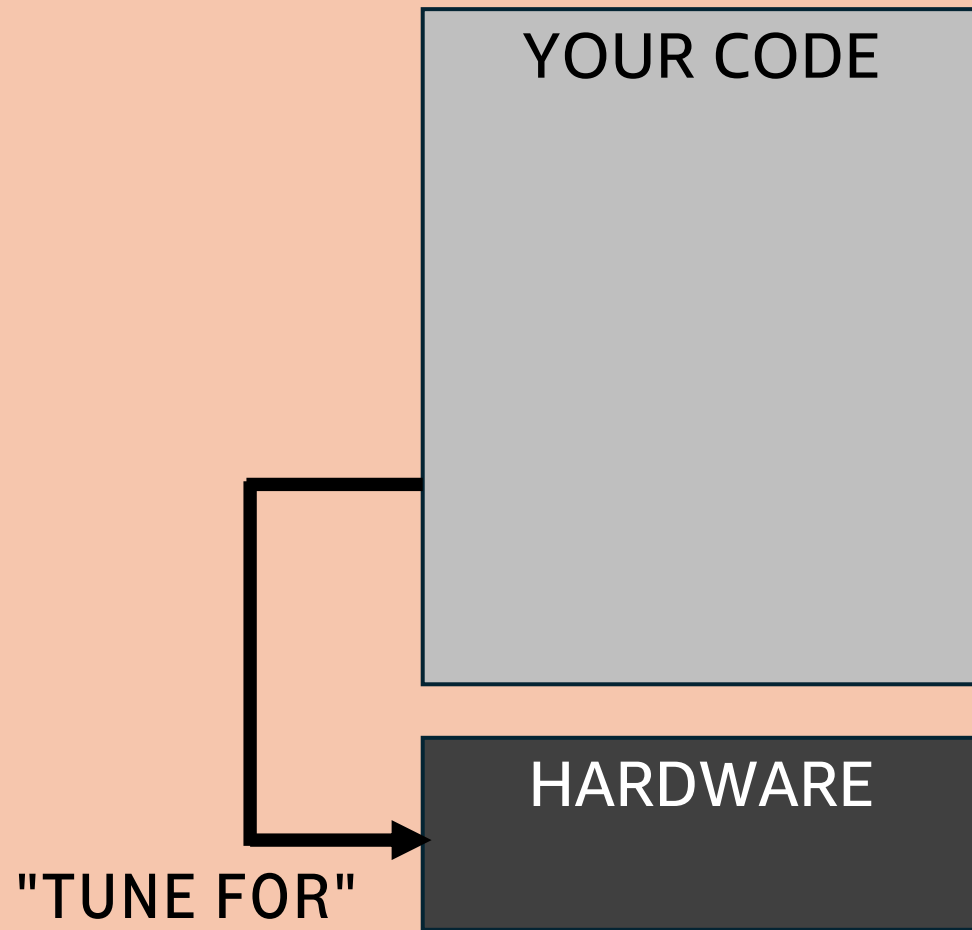
Efficiency is the new capacity.

Algorithms, like computer hardware, are a technology. Total system performance depends on choosing efficient algorithms as much **A FAST PROGRAMMING LANGUAGE**

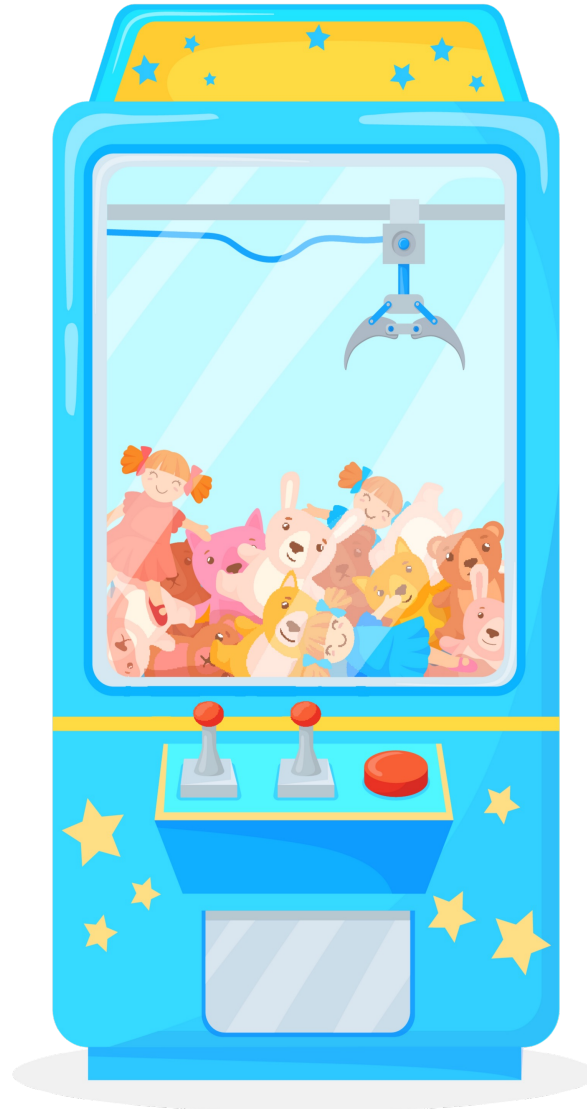
Introduction to Algorithms, Second Edition
Cormen et al.
The MIT Press



“PROGRAMMING LANGUAGE
DOESN'T MATTER”

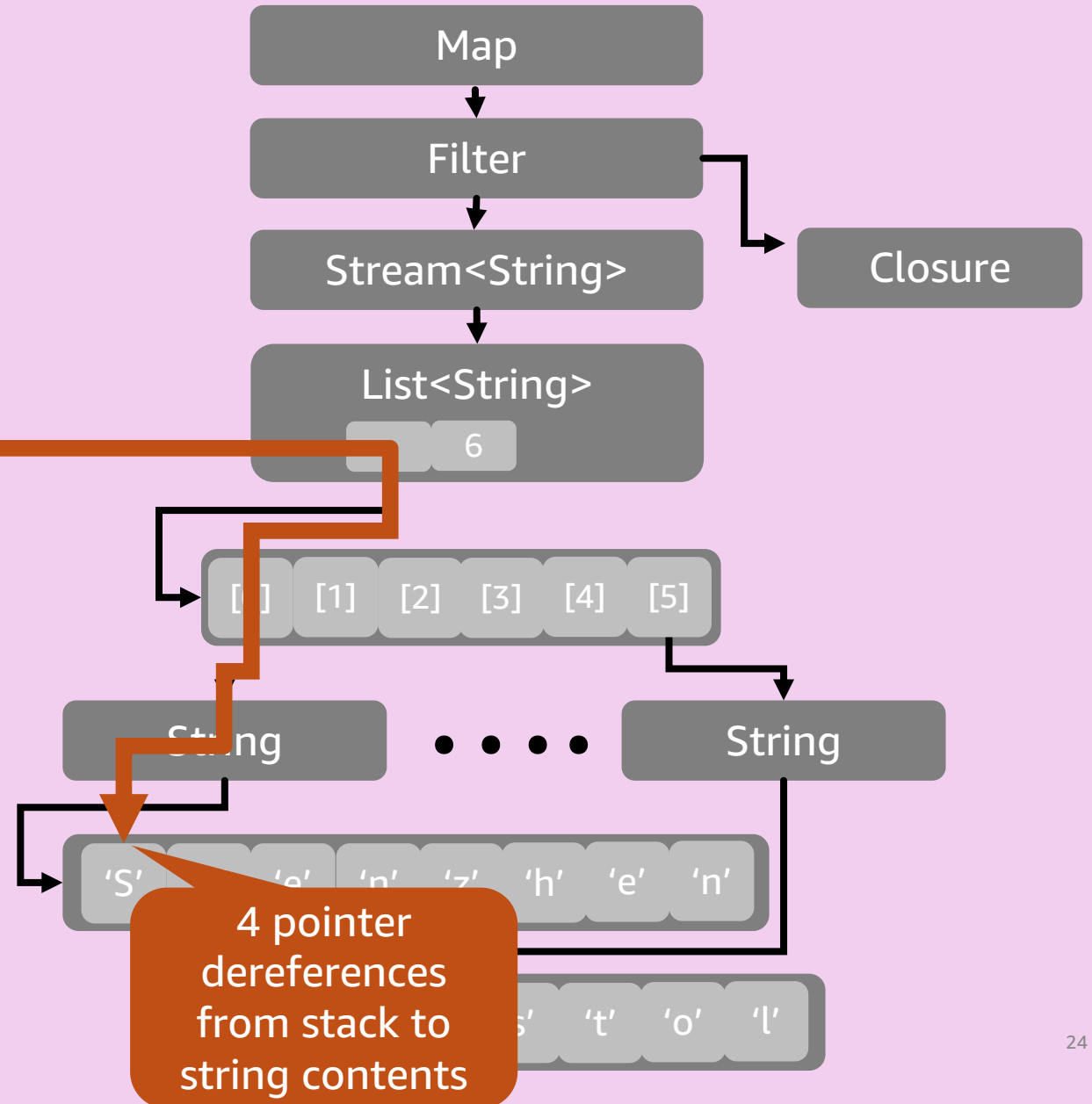
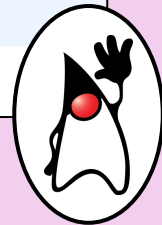


Tuning "through" the GC



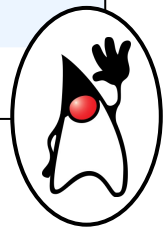
The abstraction tax

```
1 List<String> cities = Arrays.asList(  
2     "Shenzhen", "Brussels",  
3     "Taipei", "Buenos Aires",  
4     "Sydney", "Bristol",  
5 );  
6  
7 cities  
8     .stream()  
9     .filter(s -> s.startsWith("B"))  
10    .map(String::toUpperCase)  
11    .forEach(System.out::println);  
12
```



Code in Rust

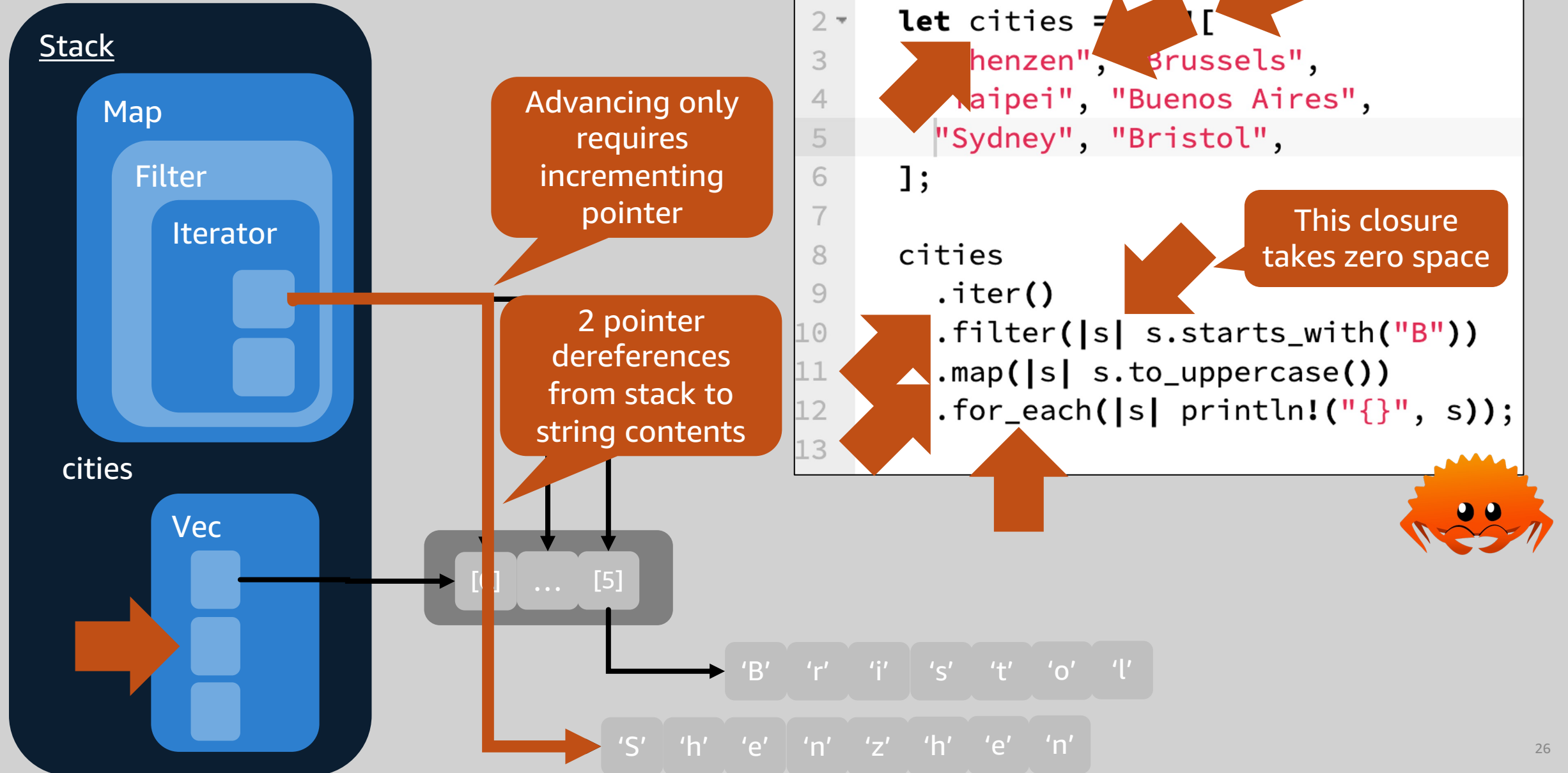
```
1 List<String> cities = Arrays.asList(  
2     "Shenzhen", "Brussels",  
3     "Taipei", "Buenos Aires",  
4     "Sydney", "Bristol",  
5 );  
6  
7 cities  
8     .stream()  
9     .filter(s -> s.startsWith("B"))  
10    .map(String::toUpperCase)  
11    .forEach(System.out::println);  
12
```



```
1 fn main() {  
2     let cities = vec![  
3         "Shenzen", "Brussels",  
4         "Taipei", "Buenos Aires",  
5         "Sydney", "Bristol",  
6     ];  
7  
8     cities  
9         .iter()  
10        .filter(|s| s.starts_with("B"))  
11        .map(|s| s.to_uppercase())  
12        .for_each(|s| println!("{}", s));  
13 }
```



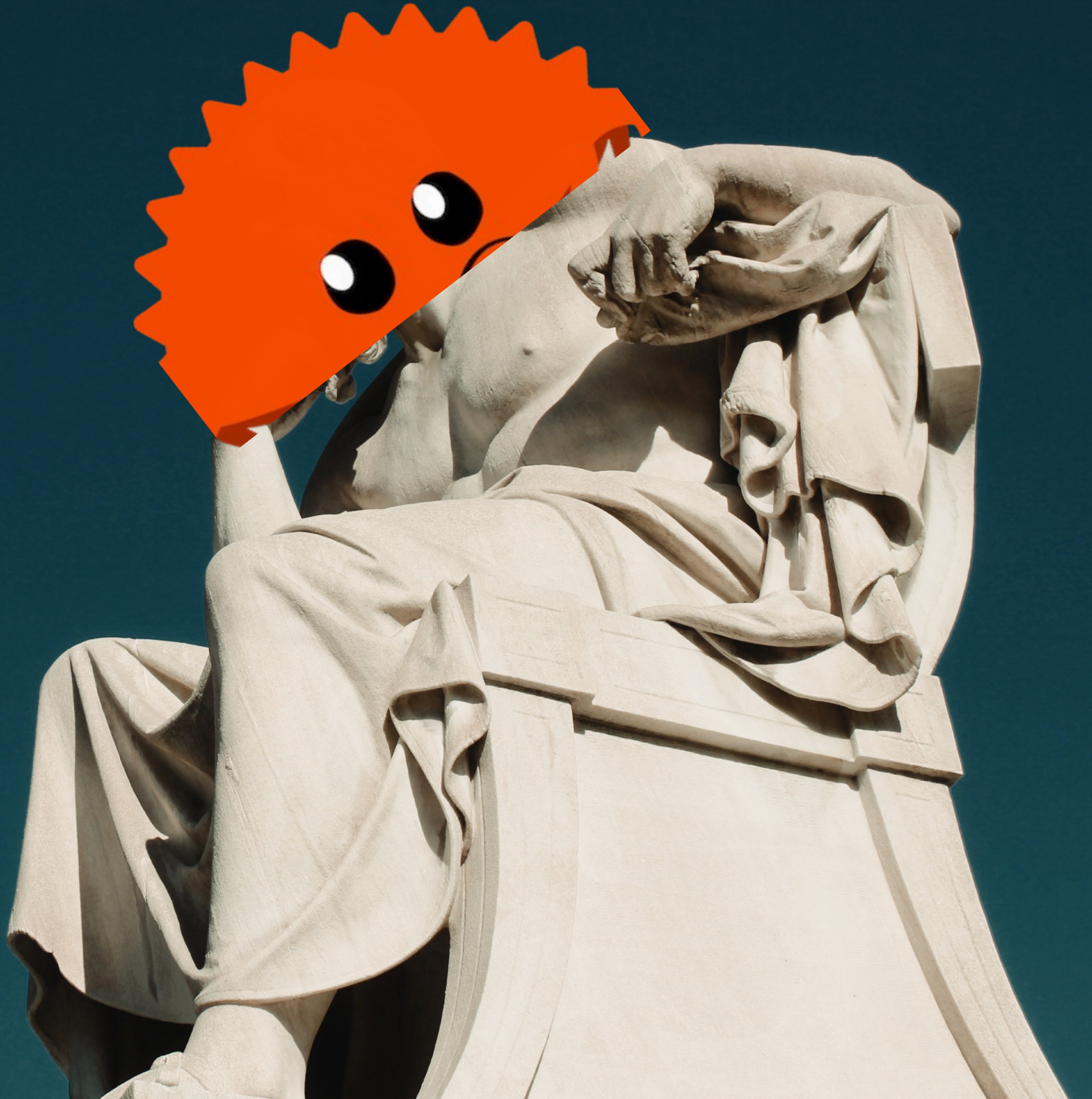
Inline layout, the stack



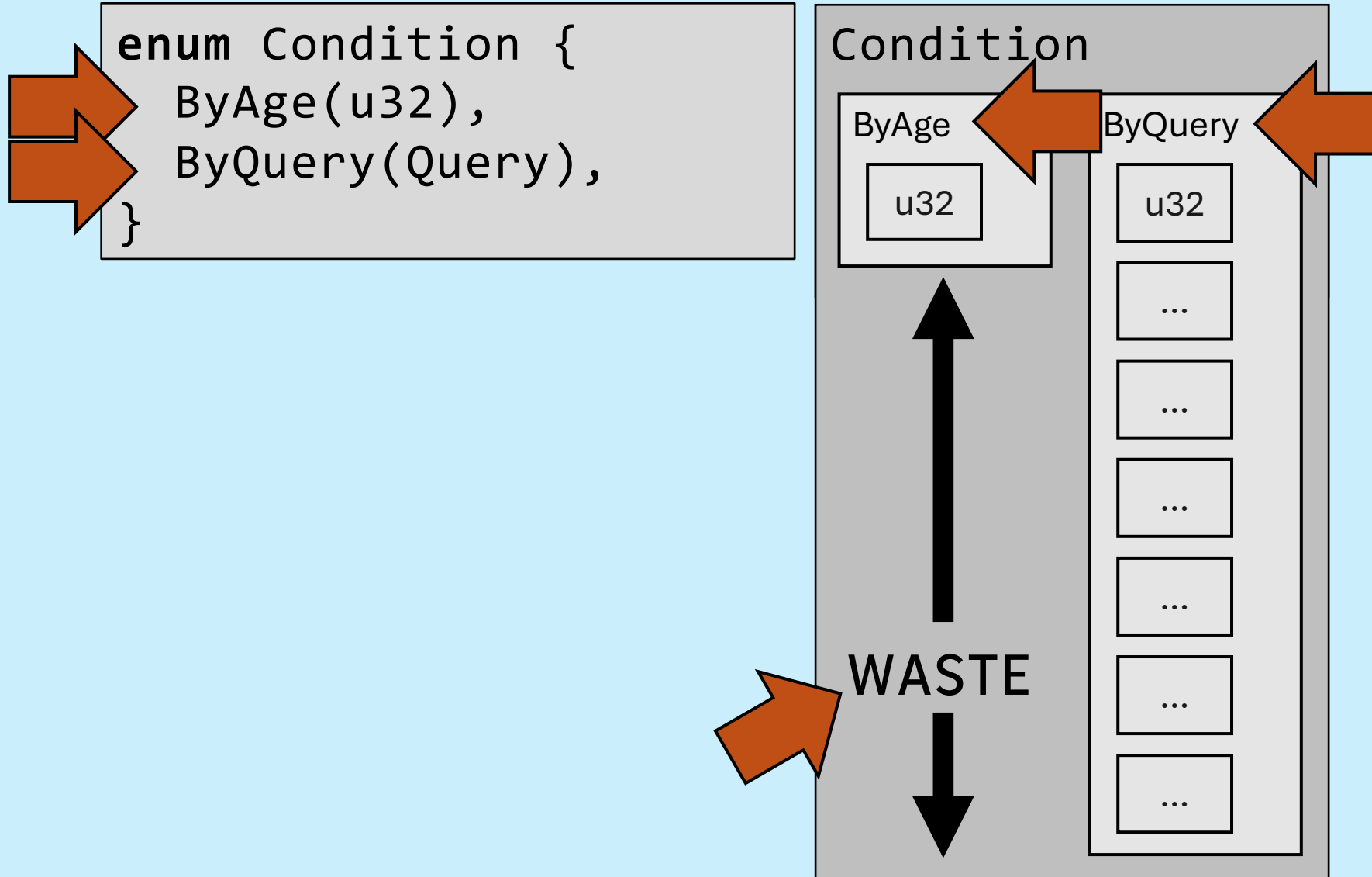
Rust Philosophy

Good performance
by default.

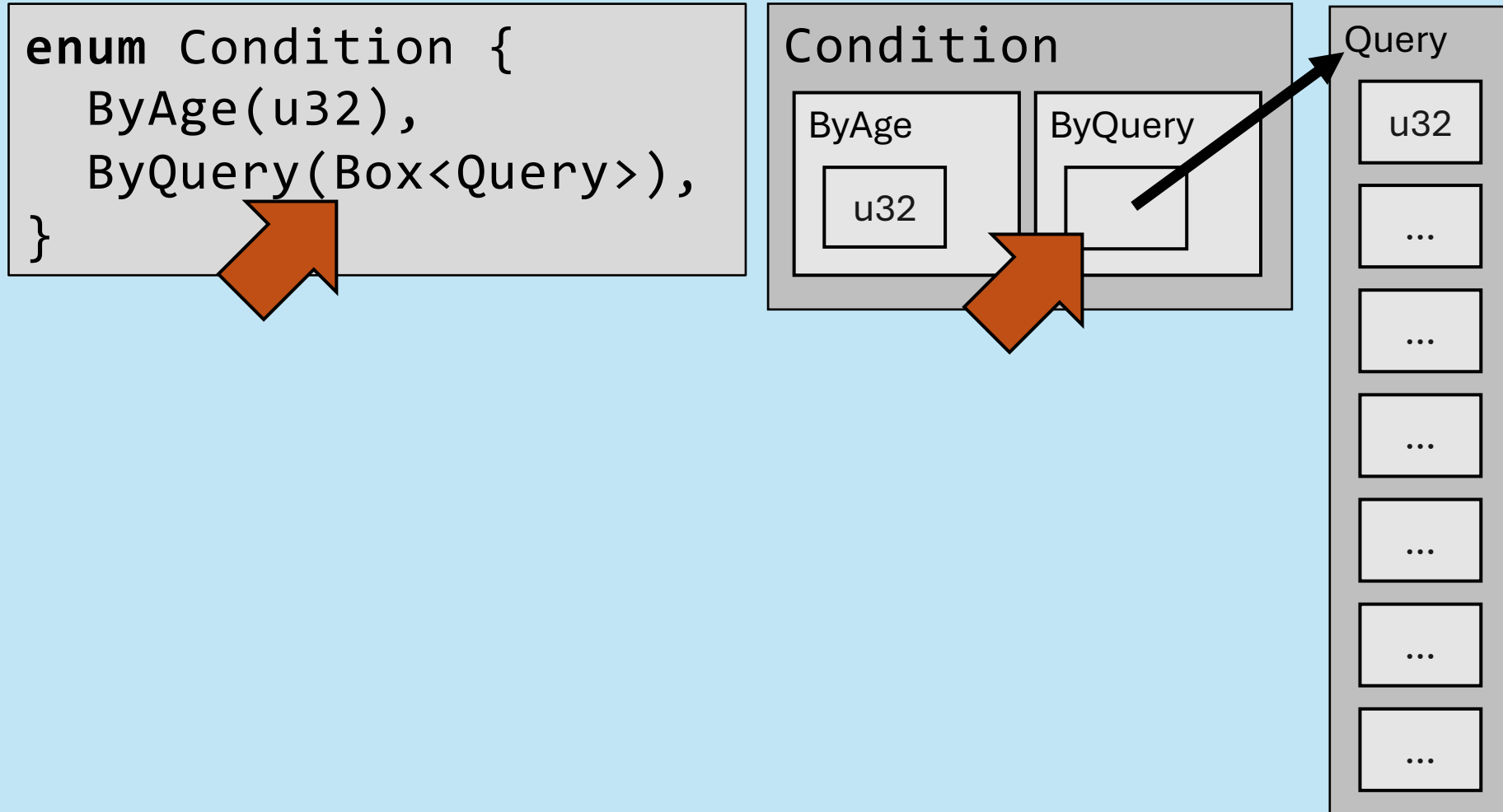
Low-level control
where needed.



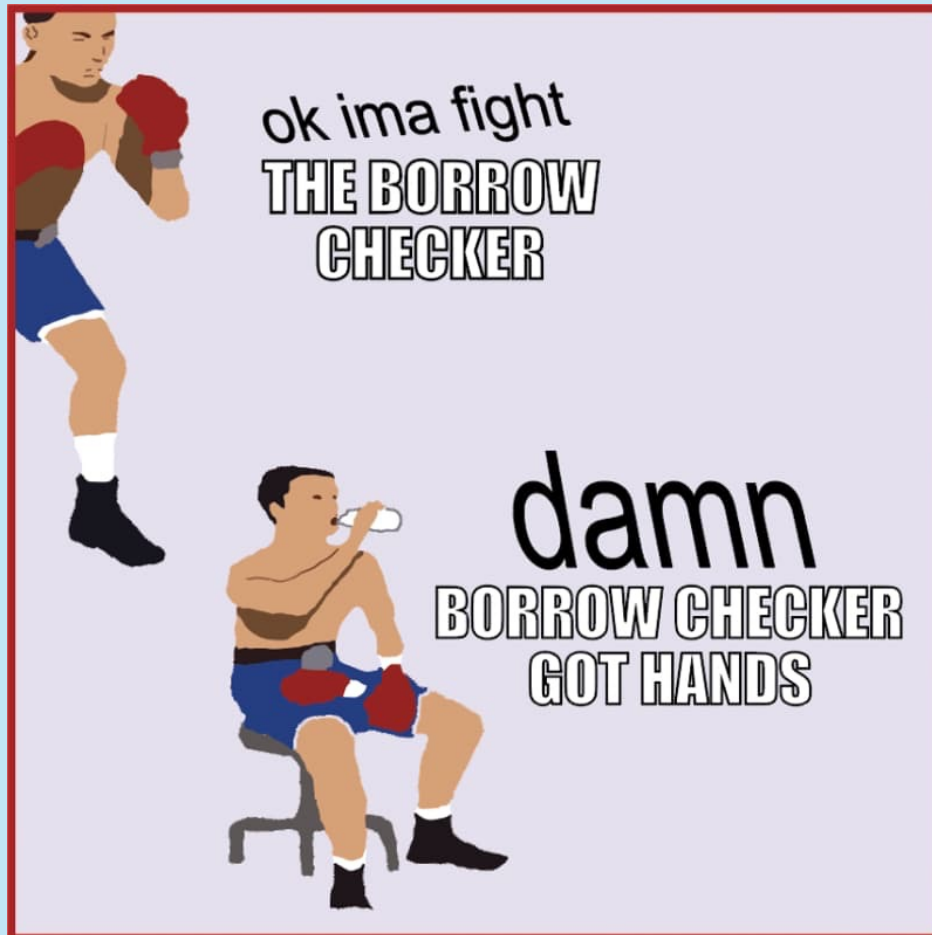
Flat layout isn't *always* good



Sometimes you want the heap



BEFORE



AFTER



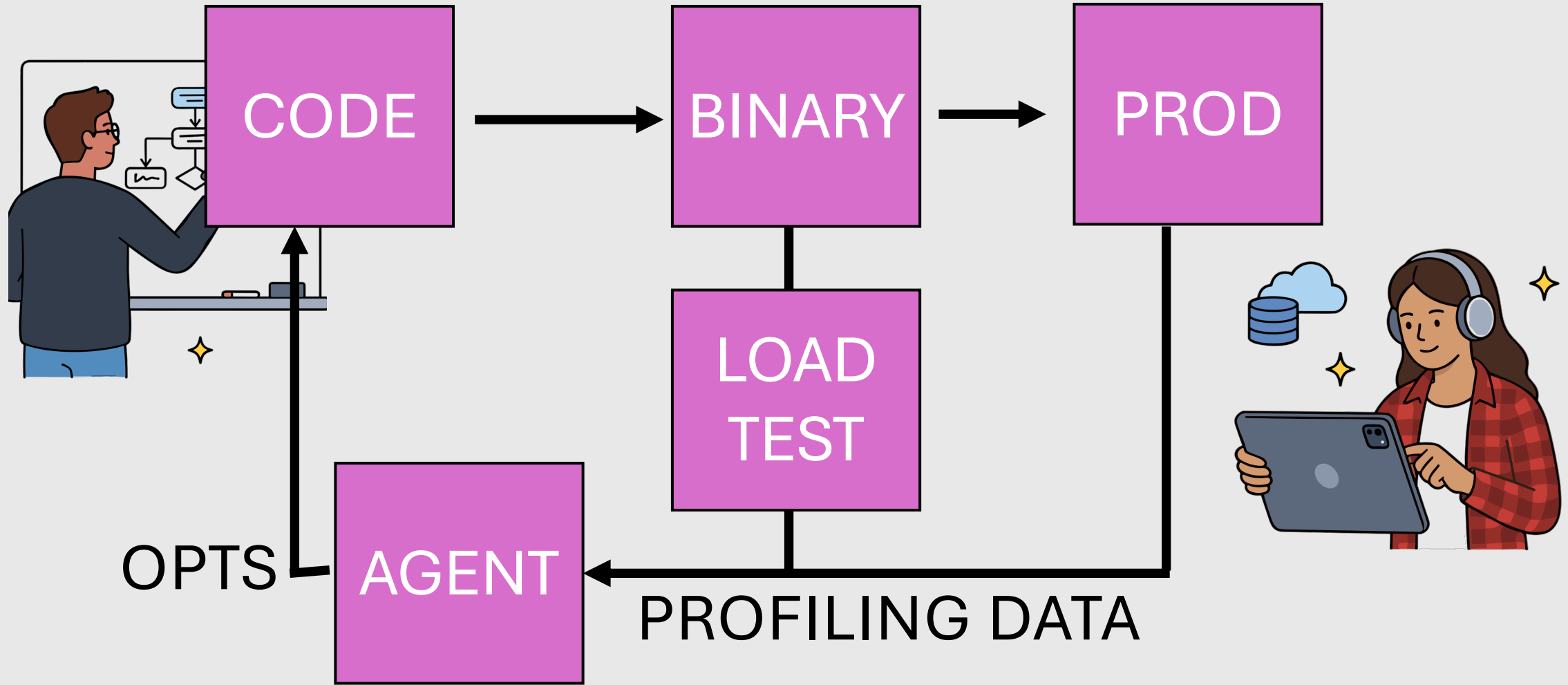
You still have to learn Rust

```
enum Condition {  
    ByAge(u32),  
    ByQuery(Box<Query>),  
}
```

**Why did you choose to
use a `Box` here?**

**Make me a website
explaining how this works.**

Goal: continuous optimization



AGENTS ARE NOT ONE-SHOT WONDERS.
THEY ARE HILL CLIMBERS.



PERF AS MEASURED
BY LOAD TESTS
AND PROD

Will this still be a problem?



I got this.

Given continuous optimization, does language matter?

Honest answer?¹
I don't know.

Let's find out!

¹ Oh my god I sound like Claude.



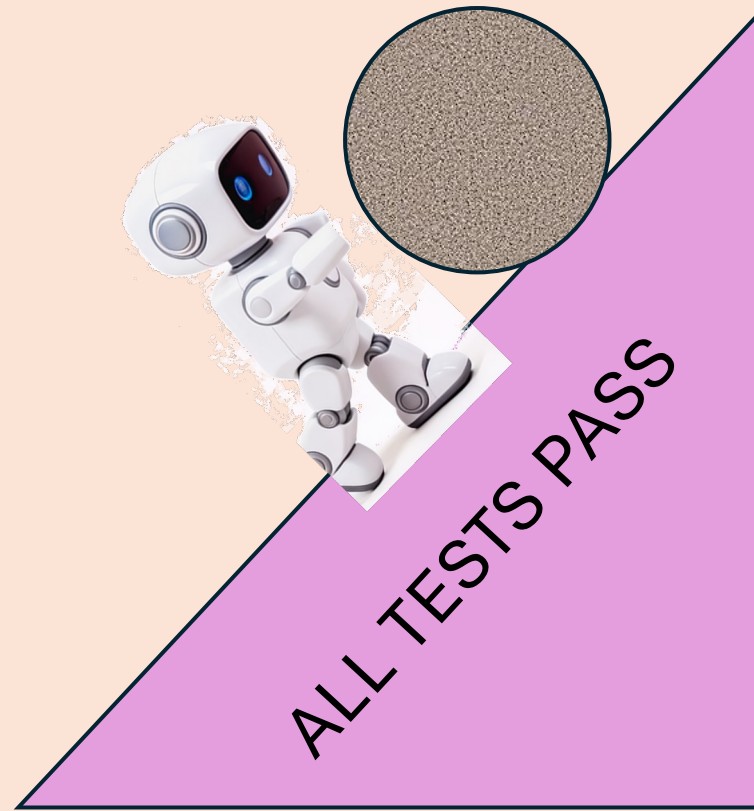
What makes Rust a good fit for agentic development?



Knobs

02

Guardrails

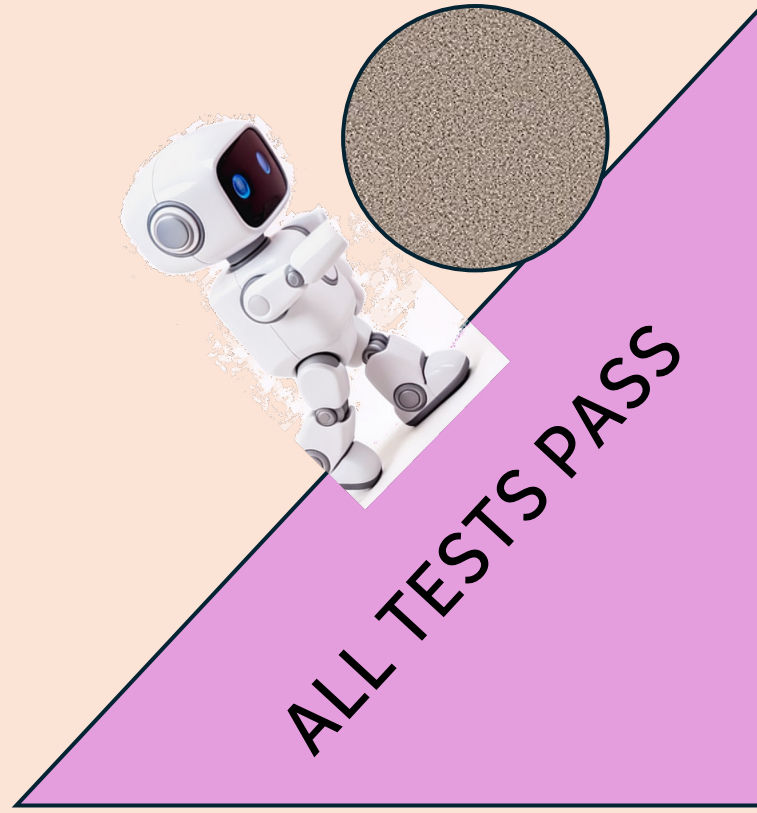
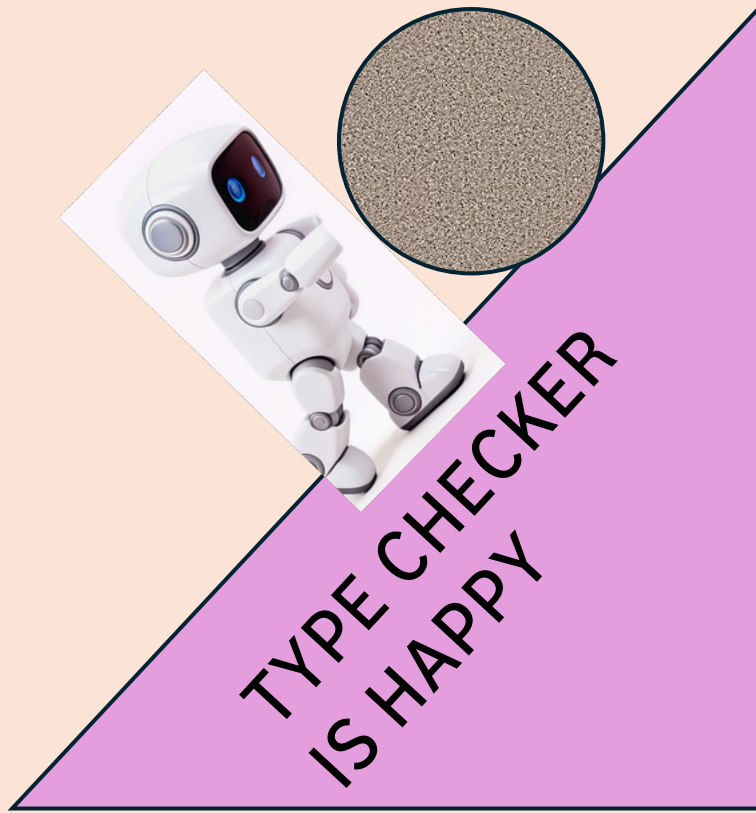


Proofs vs tests



Testing shows the
presence of bugs,
not their absence.

Edsger W. Dijkstra.



Type systems are...proofs?

Type systems are...proofs?

BEFORE

```
class Character {  
    void changeName(String n) {  
        name = n;  
    }  
}
```



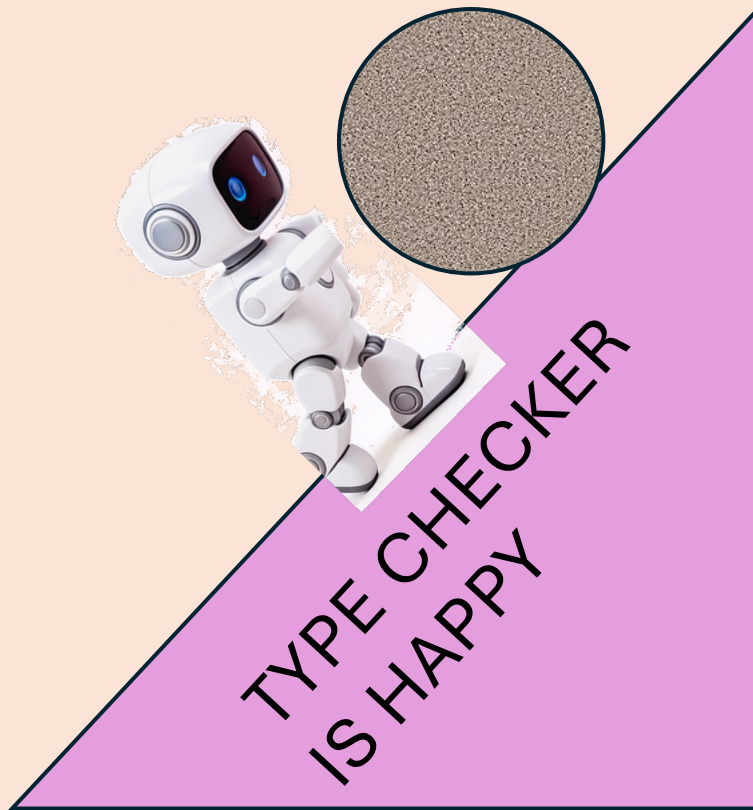
AFTER

```
class Character {  
    void setName(String n) {  
        name = n;  
    }  
}
```



“If it compiles,
will it work?”

ANSWER: PROBABLY
(DEPENDS ON REFLECTION)



All methods exist
All fields exist
etc.

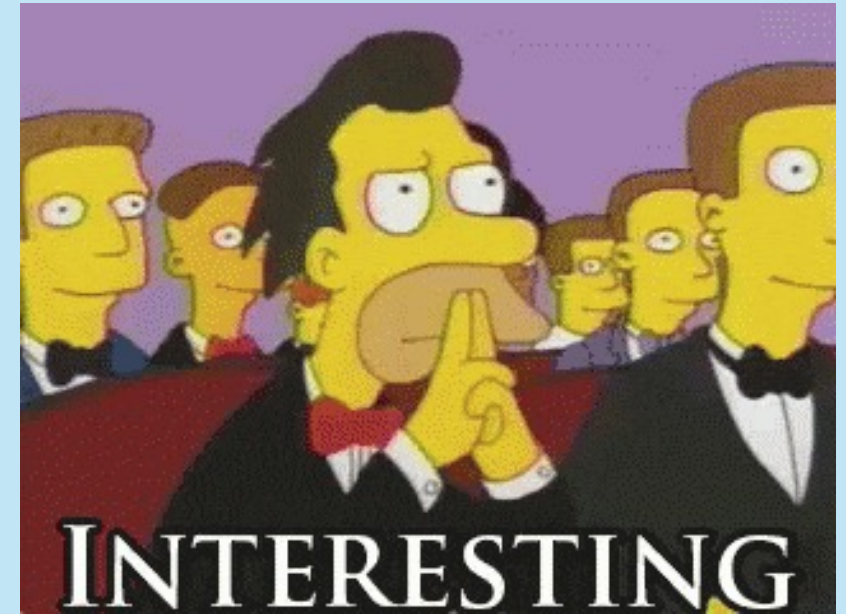
...but they have limits.

BEFORE

```
class QueryEngine {  
    void exec(DbHandle h) {  
        ...  
    }  
}
```

AFTER

```
class QueryEngine {  
    void exec(DbHandle h) {  
        ...  
        h.close();  
    }  
}
```



“If it compiles,
will it work?”

ANSWER: NO WAY TO KNOW

Is this change correct?

```
class QueryEngine {  
    void exec(DbHandle h) {  
        ...  
        h.close(); // 👉 Newly added!  
    }  
}
```

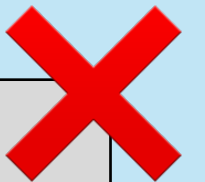
DO THE CALLERS DO THIS?



```
void do_query() {  
    let h = dbHandle();  
    query.exec(h);  
}
```



OR THIS?



```
void do_query() {  
    let h = dbHandle();  
    query.exec(h);  
    h.read();  
}
```



But wait, there's Rust!

Rust guarantees "memory safety":



- No use after free
- No use of uninitialized memory
- No data races – never forget to acquire a lock!

This requires new capabilities in the type system.

Rust's type system tracks *permissions*

```
struct QueryEngine {...}

impl QueryEngine {
    fn exec(&self, h: DbHandle) {
        h.close();
    }
}
```

Read from `self`.

Take ownership of `h`.

Give ownership of `h` away.

```
struct DbHandle {...}

impl DbHandle {
    fn read(&self) {...}
    fn close(self) {...}
}
```

Take ownership of `self`.

```
fn do_query(&self) {
    let h = self.db_handle ();
    self.query.exec(h);
    h.read();
}
```

Try to use `h` again..?

Give ownership of `h` away.

error[E0382]: borrow of moved value: `h`

Rust emphasizes reliability



No undefined behavior



No accessing shared data without a lock



No null pointers



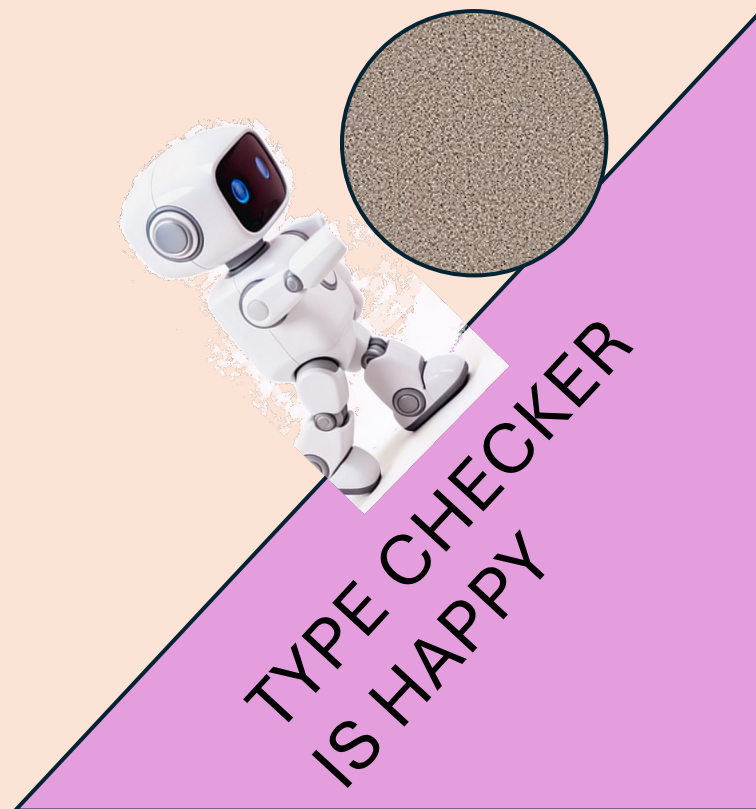
No forgotten cases, thanks to exhaustive pattern matching



No unchecked error conditions



Just THINKING about all these bugs makes me grouchy.



All methods exist
All fields exist
etc.

No data races
No resources used after free
All errors checked
etc.

Most languages



But can we do even better?

BEFORE

```
impl Preferences {  
  fn set(u: String, p: String) {  
    // ...  
  }  
}
```

USER ID PRODUCT ID

AFTER

```
impl Preferences {  
  fn set(p: String, u: String) {  
    // ...  
  }  
}
```

PRODUCT ID USER ID



“If it compiles,
will it work?”

ANSWER: DEFINITELY NOT

```
struct UserId(String);  
struct ProductId(String);
```

ROUGHLY
EQUIVALENT TO...

```
struct UserId {  
  raw: String  
}  
  
struct ProductId {  
  raw: String  
}
```

Now...

BEFORE

```
impl Preferences {  
  fn set(u: UserId, p: ProductId) {  
    // ...  
  }  
}
```



AFTER

```
impl Preferences {  
  fn set(p: ProductId, u: UserId) {  
    // ...  
  }  
}
```



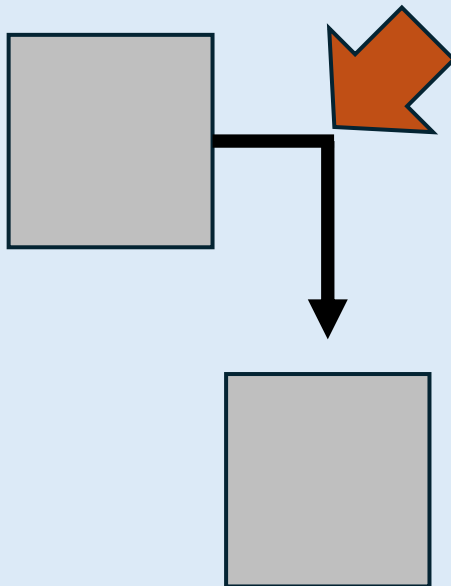
“If it compiles,
will it work?”

ANSWER: YEP

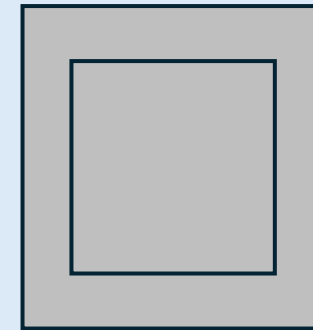
Efficiency enables reliability

```
class UserId {  
    String raw;  
}
```

```
struct UserId {  
    raw: String  
}
```



WHEN OPTIMIZING →
YOU'LL AVOID IT.



ZERO OVERHEAD →
ALWAYS SAFE TO USE

Yes, you CAN have nice things!



Reliability enables efficiency

“We had a lot of ideas to improve performance, but we were hesitant to introduce them into critical systems given the risk of subtle bugs.

“With Rust, we achieved double-digit percentage performance improvements.

Rust’s type system gave us the confidence to make those optimizations without worry.”

Seth Markle

S3 Senior Principal Engineer



Type systems vs tests



Testing shows the presence of bugs, not their absence.

Edsger W. Dijkstra.

Beware of bugs in the above code; I have only proved it correct, not tried it.

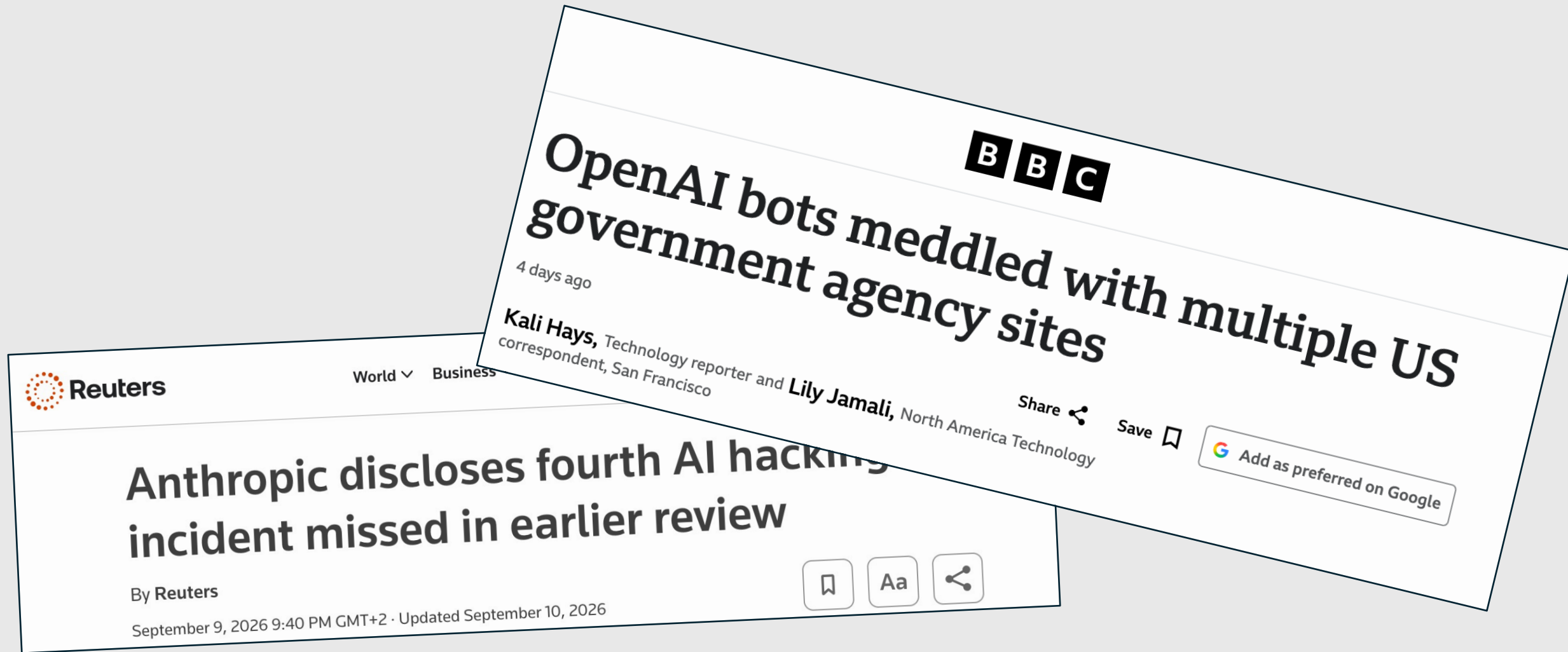
Donald Knuth

Type system can get us more assurance—

and with Rust, check resources and state.



Agents don't know when to quit



Clever, clever agents

```
struct UserId(String);
```

```
impl UserId {
```

```
fn new(s: String) -> UserId
```

```
fn as_raw(&self) -> String
```

```
}
```

```
struct ProductId(String);
```

```
impl ProductId {
```

```
fn new(s: String) -> ProductId
```

```
fn as_raw(&self) -> String
```

```
}
```

AGENT:

```
UserId::new(p_id.as_raw())
```

HUMAN:

Let's see... I've got a
`ProductId` and I
need a `UserId` ...



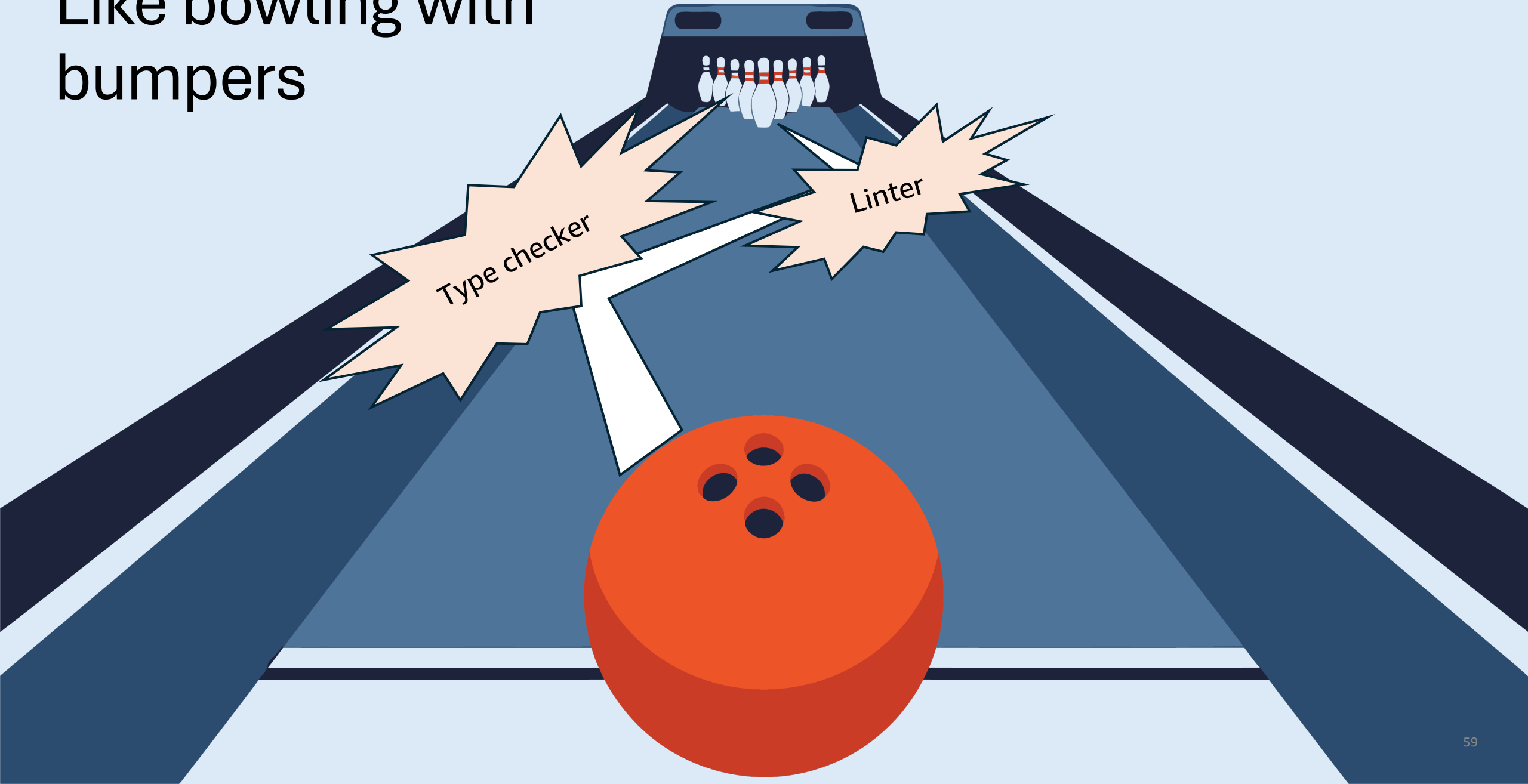
"Clippy" to the rescue!

```
clippy.toml:

[[disallowed-methods]]
path = "UserId::as_raw"
reason = """
    Don't access private representation.

    Prefer instead to use ...details here...
    """
```

Like bowling with
bumpers



What makes Rust a good fit for agentic development?



Knobs



Guardrails



...MOST OF WHICH WE
LEARNED FROM HERE.

...MOST OF WHICH WE
LEARNED FROM HERE.

And what comes next?

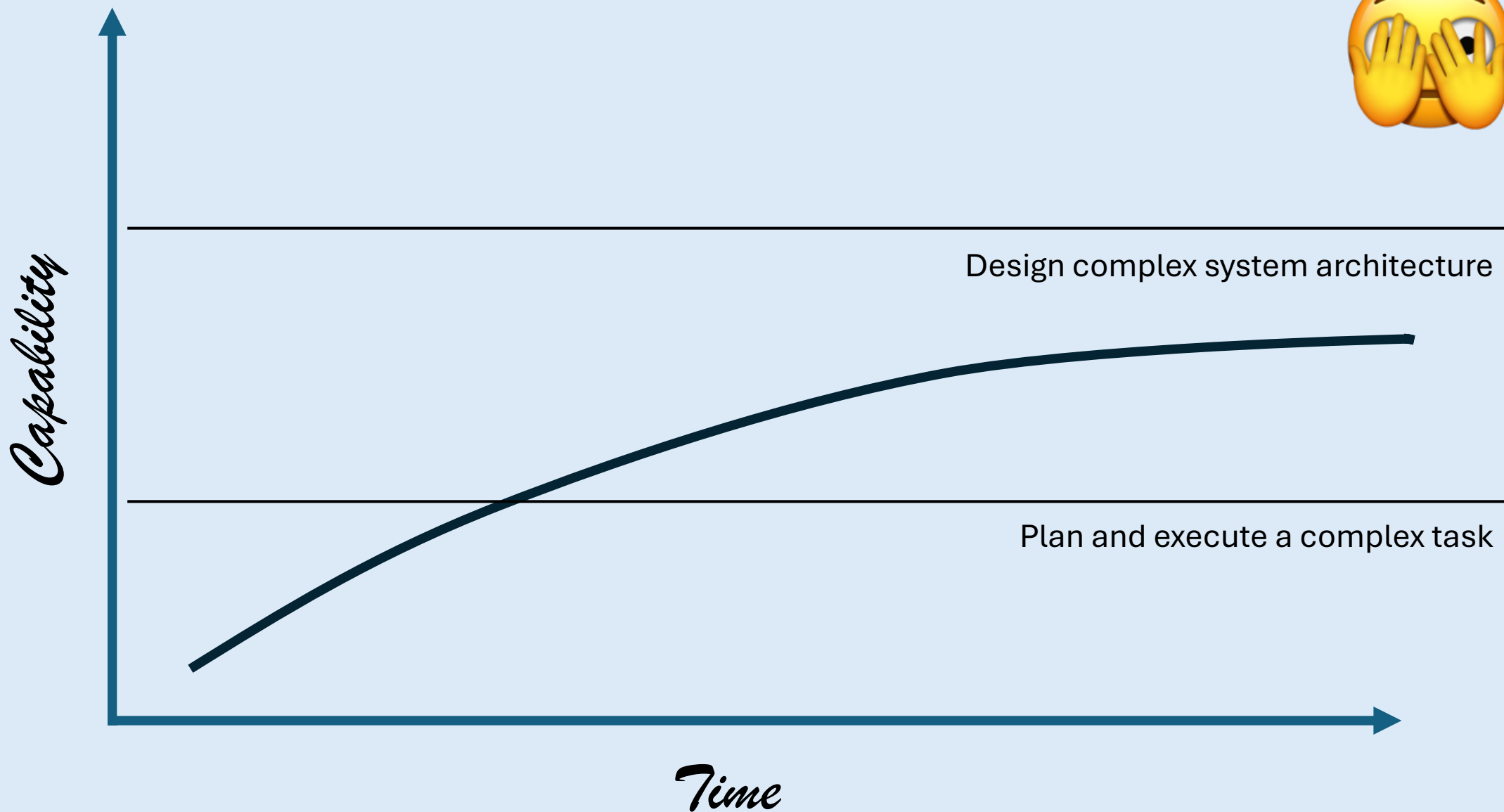


Make me a bestselling
mobile app.

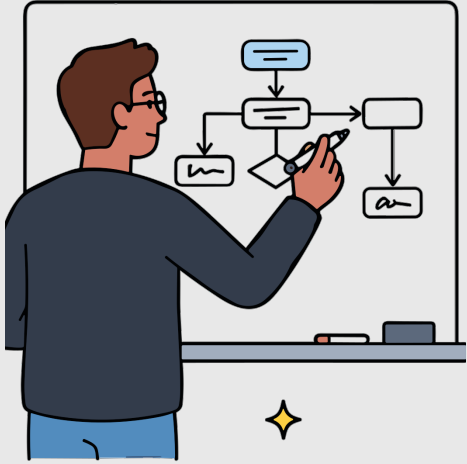
On it!







What would it take to do it for real?



Modeling

Scope

Architecture

Proof

Code

Optimize

